1. Introduction

Purpose: This document defines the test plan for the Traffic Management System (Simulation) v1.0. It outlines objectives, scope, strategy, resources, schedule, and responsibilities for testing the C-based queue simulation.

Scope: Testing covers simulation features such as vehicle generation, queue management (FIFO lane tracking), traffic signal control transitions, configuration parameter management, and traffic statistics reporting. Real-world hardware integration, physical traffic lights, cameras, road sensors, IoT traffic-control devices, and external network communications aren't included.

References: Traffic Management System SRS v1.0.

Definitions:

- **TMS:** Traffic Management System
- **FIFO:** First-In-First-Out
- **SRS:** Software Requirements Specification
- **RTM:** Requirements Traceability Matrix

2. Test Items

- Vehicle Generation module (VehicleGenerator)
- Queue Management module (QueueManager)
- Signal Control module (SignalController)
- Simulation Reporting & Statistics module (ReportGenerator)
- Simulation Configuration & Control module (ConfigManager, SimulationController)
- Security & Audit Logging module (Logger)
- Console User Interface and Menu module

6. Test Environment

- **Hardware:** Standard laptop/desktop PC with keyboard input, display monitor, CPU, and standard RAM (no specialized traffic hardware, cameras, or IoT devices required).
- **Software:** Traffic Management System application executable (v1.0), standard C runtime environment, OS (Windows, Linux, or macOS).
- **Tools:** GCC (GNU Compiler Collection) build tools, standard terminal/console shell (Bash, PowerShell, or CMD), standard C libraries (stdio.h, stdlib.h, time.h, string.h), and local text editors for config file verification.
- **Test Data:** Synthetic parameter configuration files (arrival rate λ, signal cycle durations, lane counts, maximum queue capacities) and seeded random number generator inputs for reproducible vehicle arrival patterns.

7. Test Schedule

Milestones:

- **Test Plan & Test Case Specification Design:** 16-Sep-2026 – 18-Sep-2026
- **Test Environment Setup & Synthetic Data Preparation:** 19-Sep-2026 – 20-Sep-2026
- **Unit & Module Test Execution (QueueManager, SignalController, VehicleGenerator):** 21-Sep-2026 – 22-Sep-2026
- **System & Integration Test Execution (End-to-End Simulation Loop):** 23-Sep-2026 – 24-Sep-2026
- **Security Validation & NFR Performance Testing:** 25-Sep-2026
- **Defect Verification & Retesting:** 26-Sep-2026
- **Final Test Summary Report & Course Sign-off:** 27-Sep-2026 – 28-Sep-2026

8. Test Deliverables

- **Software Test Plan Document:** The finalized STP document defining testing scope, strategy, and environmental configurations.
- **Test Case Specifications:** Fully populated test cases covering functional, non-functional, and security requirements.
- **Synthetic Test Datasets:** Configuration files (.cfg) containing pre-set arrival rates, lane capacities, signal timings, and seeded RNG values for deterministic test runs.
- **Unit Test Drivers & Harnesses:** C-based test harnesses to execute isolated module testing on QueueManager pointer logic and memory allocation.
- **Test Execution Logs:** Generated simulation execution outputs, timestamped signal transition logs, and configuration change audit logs.
- **Defect & Issue Reports:** Documented issue tracking logs identifying functional failures, memory leaks, or CLI input handling bugs.
- **Updated Requirements Traceability Matrix (RTM):** Comprehensive matrix confirming pass/fail coverage status for all SRS requirement IDs (TFS-F, TFS-NFR, TFS-S).
- **Final Test Summary Report:** Summarized test metrics, pass/fail percentages, and readiness sign-off prior to submission.

10. Risks and Mitigation

Risk	Mitigation
Memory Management Failures: Segmentation faults or memory leaks occurring during dynamic queue allocation (enqueue/dequeue) in C.	Enforce strict pointer bounds checking and run early module tests using memory debugging tools (e.g., Valgrind or GDB).
Non-Deterministic Simulation Bugs: Random arrival rates (Poisson distribution) making defects difficult to reproduce.	Implement a configurable static seed for the random number generator (time.h) to ensure test runs are 100% reproducible.
Integration Delays: Delay in merging the SignalController timing logic with the QueueManager state logic.	Define strict C function signatures (API) early and use stub functions to test modules in isolation before full integration.
Cross-Platform Compilation Issues: The C executable behaves differently on Windows vs. Linux/macOS.	Standardize the GCC compiler version and compile with strict flags (e.g., -Wall, -Wextra) across all testing environments.

11. Assumptions & Dependencies

- **Assumptions:**
    - The simulation will strictly execute on a single thread, and concurrent multithreading edge cases do not need to be tested.
    - The underlying Operating System and standard C libraries (stdio.h, stdlib.h, time.h) will execute reliably without external environment failure.
    - Test data (e.g., synthetic configuration files for arrival rates and signal timings) will be prepared prior to the execution phase.
- **Dependencies:**
    - Successful compilation of the core C modules (VehicleGenerator, QueueManager, SignalController) via GCC is required before integration testing can begin.
    - Local read/write file permissions must be granted by the OS to validate configuration and audit logging features.

12. Suspension & Resumption Criteria

Suspend testing if:

- The primary build fails to compile on the target test environment.
- Critical segmentation faults or memory corruption block more than 30% of the planned queue operation test cases.
- The system cannot continuously run without crashing due to malformed input handling failures.

Resume testing if:

- Blocking memory leaks or segmentation faults are debugged and patched by the development team.
- The application achieves a stable build that compiles cleanly and successfully launches the console UI.

14. Test Metrics & Reporting

Metrics Collected:

- **Test Case Execution Rate:** Percentage of planned test cases executed against the target C simulation build (Executed Test Cases / Total Planned Test Cases * 100).
- **Pass / Fail Rate:** Percentage of executed test cases resulting in a pass vs. fail status (Passed Test Cases / Total Executed * 100).
- **Requirement Coverage:** Percentage of SRS requirement IDs (TFS-F, TFS-NFR, TFS-S) mapped to successfully passed test cases in the RTM.
- **Defect Density & Severity:** Total number of defects identified per module (QueueManager, SignalController, VehicleGenerator), categorized by severity (Critical/Blocking, High, Medium, Low).
- **Defect Aging:** Time elapsed between defect reporting and developer resolution/patch verification.

Reports:

- **Daily Execution Status Report:** Short daily progress update detailing completed test runs, current pass rates, and newly logged memory leak or functionality defects during the execution phase.
- **Final Test Summary Report:** Comprehensive summary document compiled at the end of testing detailing final execution metrics, overall system quality, remaining open non-critical risks, and final release recommendation.

15. Approvals

Role	Name	Signature / Date
QA Lead / Test Lead	Harini	_____ / ____-09-2026
Test Engineer	Neal	_____ / ____-09-2026
Course Coordinator / Evaluator	Dr. Sheela Devi	_____ / ____-09-2026